

Algorithm Implementation/Strings/Longest common subsequence

Contents

- 1 C#
 - 1.1 Finding the LCS
- 2 Common Lisp
- 3 Python
 - 3.1 Computing the length of the LCS
 - 3.2 Reading out an LCS
 - 3.3 Reading out all LCSs
 - 3.4 Usage example
 - 3.5 Print the diff
 - 3.6 Usage example
- 4 VB.NET
- 5 Java
- 6 C++
- 7 Ruby
- 8 JavaScript
- 9 APL
 - 9.1 Computing the length of the LCS
 - 9.2 Reading out all LCSs
 - 9.3 Usage example
- 10 J

C#

Finding the LCS

The following C# program calculates the longest common subsequence (note the singular) of 2 strings. For example, for the strings "computer" and "houseboat" this algorithm returns a value of 3, specifically the string "out".

```
public static int GetLCS(string str1, string str2)
{
    int[,] table;
    return GetLCSInternal(str1, str2, out table);
}

private static int GetLCSInternal(string str1, string str2, out int[,] matrix)
{
    matrix = null;

    if (string.IsNullOrEmpty(str1) || string.IsNullOrEmpty(str2))
    {
        return 0;
    }
}
```

```

int[,] table = new int[str1.Length + 1, str2.Length + 1];
for (int i = 0; i < table.GetLength(0); i++)
{
    table[i, 0] = 0;
}
for(int j= 0;j<table.GetLength(1); j++)
{
    table[0,j] = 0;
}

for (int i = 1; i < table.GetLength(0); i++)
{
    for (int j = 1; j < table.GetLength(1); j++)
    {
        if (str1[i-1] == str2[j-1])
            table[i, j] = table[i - 1, j - 1] + 1;
        else
        {
            if (table[i, j - 1] > table[i - 1, j])
                table[i, j] = table[i, j - 1];
            else
                table[i, j] = table[i - 1, j];
        }
    }
}

matrix = table;
return table[str1.Length, str2.Length];
}

```

//Reading Out ALL LCS sorted in Lexicographic order

```

using System;
using System.Collections.Generic;
using System.Collections;
using System.Linq;
using System.Text;

namespace LambdaPractice
{
    class Program
    {
        static int[,] c;

        static int max(int a, int b)
        {
            return (a > b) ? a : b;
        }

        static int LCS(string s1, string s2)
        {
            for (int i = 1; i <= s1.Length; i++)
                c[i,0] = 0;
            for (int i = 1; i <= s2.Length; i++)
                c[0, i] = 0;

            for (int i=1;i<=s1.Length;i++)
                for (int j = 1; j <= s2.Length; j++)
                {
                    if (s1[i-1] == s2[j-1])
                        c[i, j] = c[i - 1, j - 1] + 1;
                    else
                    {
                        c[i, j] = max(c[i - 1, j], c[i, j - 1]);
                    }
                }

            return c[s1.Length, s2.Length];
        }

        /* Prints one LCS
        static string BackTrack(string s1, string s2, int i, int j)
        {
            if (i == 0 || j == 0)
                return "";
            if (s1[i - 1] == s2[j - 1])

```

```

        return BackTrack(s1, s2, i - 1, j - 1) + s1[i - 1];
    else if (c[i - 1, j] > c[i, j - 1])
        return BackTrack(s1, s2, i - 1, j);
    else
        return BackTrack(s1, s2, i, j - 1);

}*/

static SortedSet<string> backtrack(string s1, string s2, int i, int j)
{
    if (i == 0 || j == 0)
        return new SortedSet<string>(){""} ;
    else if (s1[i - 1] == s2[j - 1])
    {
        SortedSet<string> temp = new SortedSet<string>();
        SortedSet<string> holder = backtrack(s1, s2, i - 1, j - 1);
        if (holder.Count == 0)
        {
            temp.Add(s1[i - 1]);
        }
        foreach (string str in holder)
            temp.Add(str + s1[i - 1]);

        return temp;
    }
    else
    {
        SortedSet<string> Result = new SortedSet<string>() ;
        if (c[i - 1, j] >= c[i, j - 1])
        {
            SortedSet<string> holder = backtrack(s1, s2, i - 1, j);

            foreach (string s in holder)
                Result.Add(s);
        }

        if (c[i, j - 1] >= c[i - 1, j])
        {
            SortedSet<string> holder = backtrack(s1, s2, i, j - 1);

            foreach (string s in holder)
                Result.Add(s);
        }

        return Result;
    }
}

static void Main(string[] args)
{
    string s1, s2;
    s1 = Console.ReadLine();
    s2 = Console.ReadLine();
    c = new int[s1.Length+1, s2.Length+1];
    LCS(s1, s2);
    // Console.WriteLine(BackTrack(s1, s2, s1.Length, s2.Length));
    // Console.WriteLine(s1.Length);
    SortedSet<string> st = backtrack(s1, s2, s1.Length, s2.Length);

    foreach (string str in st)
        Console.WriteLine(str);
    GC.Collect();

    Console.ReadLine();
}
}
}

```

Common Lisp

This version only works for lists but can be generalized for all sequences.

```
(defun lcs-list (list-1 list-2 &key (test #'eql))
  "Find the longest common subsequence of LIST-1 and LIST-2 using TEST."
  (cond
    ((null list-1) nil)
    ((null list-2) nil)
    ((funcall test (first list-1) (first list-2))
     (cons (first list-1) (lcs-list (rest list-1) (rest list-2) :test test)))
    (t (let ((lcs-1 (lcs-list list-1 (rest list-2) :test test))
              (lcs-2 (lcs-list (rest list-1) list-2 :test test)))
          (if (> (length lcs-1) (length lcs-2))
              lcs-1
              lcs-2))))))

(defun diff (list1 list2 &key (test #'eql))
  "Find the differences between LIST1 and LIST2 using TEST."
  (let ((lcs (lcs-list list1 list2 :test test))
        result)
    (dolist (c lcs)
      (let* ((sync-list1 (position c list1 :test test))
             (sync-list2 (position c list2 :test test))
             (removed (subseq list1 0 sync-list1))
             (added (subseq list2 0 sync-list2)))
        (setf list1 (subseq list1 (1+ sync-list1)))
        (setf list2 (subseq list2 (1+ sync-list2)))
        (when removed
          (push (cons :removed removed) result))
        (when added
          (push (cons :added added) result))
        (push c result)))
    (when list1
      (push (cons :removed list1) result))
    (when list2
      (push (cons :added list2) result))
    (nreverse result)))
```

Python

Computing the length of the LCS

```
def LCS(X, Y):
    m = len(X)
    n = len(Y)
    # An (m+1) times (n+1) matrix
    C = [[0 for j in range(n+1)] for i in range(m+1)]
    for i in range(1, m+1):
        for j in range(1, n+1):
            if X[i-1] == Y[j-1]:
                C[i][j] = C[i-1][j-1] + 1
            else:
                C[i][j] = max(C[i][j-1], C[i-1][j])
    return C
```

Reading out an LCS

```
def backTrack(C, X, Y, i, j):
    if i == 0 or j == 0:
        return ""
    elif X[i-1] == Y[j-1]:
        return backTrack(C, X, Y, i-1, j-1) + X[i-1]
    else:
        if C[i][j-1] > C[i-1][j]:
            return backTrack(C, X, Y, i, j-1)
        else:
            return backTrack(C, X, Y, i-1, j)
```

```
return backTrack(C, X, Y, i-1, j)
```

Reading out all LCSs

```
def backTrackAll(C, X, Y, i, j):
    if i == 0 or j == 0:
        return set("")
    elif X[i-1] == Y[j-1]:
        return set([Z + X[i-1] for Z in backTrackAll(C, X, Y, i-1, j-1)])
    else:
        R = set()
        if C[i][j-1] >= C[i-1][j]:
            R.update(backTrackAll(C, X, Y, i, j-1))
        if C[i-1][j] >= C[i][j-1]:
            R.update(backTrackAll(C, X, Y, i-1, j))
        return R
```

Usage example

```
X = "AATCC"
Y = "ACACG"
m = len(X)
n = len(Y)
C = LCS(X, Y)

print "Some LCS: '%s'" % backTrack(C, X, Y, m, n)
print "All LCSs: %s" % backTrackAll(C, X, Y, m, n)
```

It prints the following:

```
Some LCS: 'AAC'
All LCSs: set(['ACC', 'AAC'])
```

Print the diff

```
def printDiff(C, X, Y, i, j):
    if i > 0 and j > 0 and X[i-1] == Y[j-1]:
        printDiff(C, X, Y, i-1, j-1)
        print " " + X[i-1]
    else:
        if j > 0 and (i == 0 or C[i][j-1] >= C[i-1][j]):
            printDiff(C, X, Y, i, j-1)
            print "+" + Y[j-1]
        elif i > 0 and (j == 0 or C[i][j-1] < C[i-1][j]):
            printDiff(C, X, Y, i-1, j)
            print "-" + X[i-1]
```

Usage example

```
X = [
```

```

    "This part of the document has stayed",
    "the same from version to version.",
    "",
    "This paragraph contains text that is",
    "outdated - it will be deprecated ''and''",
    "deleted ''in'' the near future.",
    "",
    "It is important to spell check this",
    "dokument. On the other hand, a misspelled",
    "word isn't the end of the world.",
]
Y = [
    "This is an important notice! It should",
    "therefore be located at the beginning of",
    "this document!",
    "",
    "This part of the document has stayed",
    "the same from version to version.",
    "",
    "It is important to spell check this",
    "dokument. On the other hand, a misspelled",
    "word isn't the end of the world. This",
    "paragraph contains important new",
    "additions to this document.",
]
C = LCS(X, Y)
printDiff(C, X, Y, len(X), len(Y))

```

It prints the following:

```

+ This is an important notice! It should
+ therefore be located at the beginning of
+ this document!
+
+ This part of the document has stayed
+ the same from version to version.
+
+ This paragraph contains text that is
+ outdated - it will be deprecated and
+ deleted in the near future.
+
+ It is important to spell check this
+ dokument. On the other hand, a misspelled
+ word isn't the end of the world.
+ document. On the other hand, a misspelled
+ word isn't the end of the world. This
+ paragraph contains important new
+ additions to this document.

```

VB.NET

The following VB.NET program calculates the longest common subsequence (note the singular) of 2 strings. For example, for the strings "computer" and "houseboat" this algorithm returns a value of 3, specifically the string "out".

```

Public Function LongestCommonSubsequence(ByVal s1 As String, ByVal s2 As String) As Integer

    'Bulletproofing - 1 or both inputs contains nothing
    If s1.Length.Equals(0) Or s2.Length.Equals(0) Then
        Return "0"
    End If

    '*** Actual Algorithm From Here ***
    Dim num(s1.Length - 1, s2.Length - 1) As Long           '2D Array
    Dim letter1 As Char = Nothing
    Dim letter2 As Char = Nothing

```

```

For i As Integer = 0 To s1.Length - 1
    For j As Integer = 0 To s2.Length - 1
        letter1 = s1.Chars(i)
        letter2 = s2.Chars(j)

        If letter1.Equals(letter2) Then
            If i.Equals(0) Or j.Equals(0) Then 'The first elements respectively
                num(i, j) = 1
            Else
                num(i, j) = 1 + num(i - 1, j - 1)
            End If
        Else
            If i.Equals(0) And j.Equals(0) Then
                num(i, j) = 0
            ElseIf i.Equals(0) And Not j.Equals(0) Then 'First ith element
                num(i, j) = Math.Max(0, num(i, j - 1))
            ElseIf j.Equals(0) And Not i.Equals(0) Then 'First jth element
                num(i, j) = Math.Max(num(i - 1, j), 0)
            ElseIf i <> 0 And j <> 0 Then
                num(i, j) = Math.Max(num(i - 1, j), num(i, j - 1))
            End If
        End If
    Next j
Next i

Return num(s1.Length - 1, s2.Length - 1)
End Function

```

Usage: LongestCommonSubsequence("computer", "houseboat")

Java

```

public static <E> List<E> LongestCommonSubsequence(E[] s1, E[] s2)
{
    int[][] num = new int[s1.length+1][s2.length+1]; //2D array, initialized to 0

    //Actual algorithm
    for (int i = 1; i <= s1.length; i++)
        for (int j = 1; j <= s2.length; j++)
            if (s1[i-1].equals(s2[j-1]))
                num[i][j] = 1 + num[i-1][j-1];
            else
                num[i][j] = Math.max(num[i-1][j], num[i][j-1]);

    System.out.println("length of LCS = " + num[s1.length][s2.length]);

    int s1position = s1.length, s2position = s2.length;
    List<E> result = new LinkedList<E>();

    while (s1position != 0 && s2position != 0)
    {
        if (s1[s1position - 1].equals(s2[s2position - 1]))
        {
            result.add(s1[s1position - 1]);
            s1position--;
            s2position--;
        }
        else if (num[s1position][s2position - 1] >= num[s1position][s2position])
        {
            s2position--;
        }
        else
        {
            s1position--;
        }
    }
    Collections.reverse(result);
    return result;
}

```

The Java implementation uses two classes. The first, an abstract class that implements the algorithm. The second, an concrete class implementing this for a string. Obviously, you could use this so that instead of comparing characters in a string, it could compare lines in a file, blocks of code, nodes in an XML document, or whatever you choose.

```
import static java.lang.Math.*;

import java.util.ArrayList;
import java.util.List;
import static java.lang.Math.*;
/**
 * A class to compute the longest common subsequence in two strings.
 * Algorithms from Wikipedia:
 * http://en.wikipedia.org/wiki/Longest\_common\_subsequence\_problem
 *
 * @author jhess
 */
public abstract class LongestCommonSubsequence<VALUE> {
    private int[][] c;
    private ArrayList<DiffEntry<VALUE>> diff;
    private ArrayList<VALUE> backtrack;

    /**
     * A constructor for classes inheriting this one, allowing them to
     * do some initialization before setting the values of X and Y. Once
     * the initialization is complete, the inheriting class must call
     * initValues(VALUE[] x, VALUE[] y)
     */
    protected LongestCommonSubsequence() {

    }

    protected abstract int lengthOfY() ;
    protected abstract int lengthOfX() ;
    protected abstract VALUE valueOfX(int index) ;
    protected abstract VALUE valueOfY(int index) ;

    protected boolean equals(VALUE x1, VALUE y1) {
        return (null == x1 && null == y1) || x1.equals(y1);
    }

    private boolean isXYEqual(int i, int j) {
        return equals(valueOfXInternal(i),valueOfYInternal(j));
    }

    private VALUE valueOfXInternal(int i) {
        return valueOfX(i-1);
    }

    private VALUE valueOfYInternal(int j) {
        return valueOfY(j-1);
    }

    public void calculateLcs() {
        if(c != null) {
            return;
        }
        c = new int[lengthOfX()+1][];
        for(int i = 0; i < c.length; i++) {
            c[i] = new int[lengthOfY()+1];
        }

        for(int i = 1; i < c.length; i++) {
            for(int j = 1; j < c[i].length; j++) {
                if(isXYEqual(i, j)) {
                    c[i][j] = c[i-1][j-1] + 1;
                } else {
                    c[i][j] = max(c[i][j-1], c[i-1][j]);
                }
            }
        }
    }
}
```



```

    }

    public int getLcsLength() {
        calculateLcs();

        return c[lengthOfX()][lengthOfY()];
    }

    public int getMinEditDistance() {
        calculateLcs();
        return lengthOfX()+lengthOfY()-2*abs(getLcsLength());
    }

    public List<VALUE> backtrack() {
        calculateLcs();
        if(this.backtrack == null) {
            this.backtrack = new ArrayList<VALUE>();
            backtrack(lengthOfX(),lengthOfY());
        }
        return this.backtrack;
    }

    public void backtrack(int i, int j) {
        calculateLcs();

        if (i == 0 || j == 0) {
            return;
        }
        else if (isXYEqual(i, j)) {
            backtrack(i-1,j-1);
            backtrack.add(valueOfXInternal(i));
        }
        else {
            if(c[i][j-1] > c[i-1][j]) {
                backtrack(i,j-1);
            } else {
                backtrack(i-1,j);
            }
        }
    }

    public List<DiffEntry<VALUE>> diff() {
        calculateLcs();

        if(this.diff == null) {
            this.diff = new ArrayList<DiffEntry<VALUE>>();
            diff(lengthOfX(),lengthOfY());
        }
        return this.diff;
    }

    private void diff(int i, int j) {
        calculateLcs();

        while(!(i==0 && j==0)) {
            if (i > 0 && j > 0 && isXYEqual(i, j)) {
                this.diff.add(new DiffEntry<VALUE>(DiffType.NONE,
                    valueOfXInternal(i)));
                i--;j--;
            } else {
                if (j > 0 && (i == 0 || c[i][j - 1] >= c[i - 1][j])) {
                    this.diff.add(new DiffEntry<VALUE>(DiffType.ADD,
                        valueOfYInternal(j)));
                    j--;
                } else if (i > 0 && (j == 0 || c[i][j - 1] < c[i - 1][j])) {
                    this.diff.add(new DiffEntry<VALUE>(DiffType.REMOVE,
                        valueOfXInternal(i)));
                    i--;
                }
            }
        }

        Collections.reverse(this.diff);
    }

    @Override
    public String toString() {
        calculateLcs();

```

```

        StringBuffer buf = new StringBuffer();
        buf.append(" ");
        for(int j = 1; j <= lengthOfY(); j++) {
            buf.append(valueOfYInternal(j));
        }
        buf.append("\n");
        buf.append(" ");
        for(int j = 0; j < c[0].length; j++) {
            buf.append(Integer.toString(c[0][j]));
        }
        buf.append("\n");
        for(int i = 1; i < c.length; i++) {
            buf.append(valueOfXInternal(i));
            for(int j = 0; j < c[i].length; j++) {
                buf.append(Integer.toString(c[i][j]));
            }
            buf.append("\n");
        }
        return buf.toString();
    }

    public static enum DiffType {
        ADD("+", "add"), REMOVE("-", "remove"), NONE(" ", "none");

        private String val;
        private String name;

        DiffType(String val, String name) {
            this.val = val;
            this.name = name;
        }

        @Override
        public String toString() {
            return val;
        }

        public String getName() {
            return name;
        }

        public String getVal() {
            return val;
        }
    }

    public static class DiffEntry<VALUE> {
        private DiffType type;
        private VALUE value;

        public DiffEntry(DiffType type, VALUE value) {
            super();
            this.type = type;
            this.value = value;
        }

        public DiffType getType() {
            return type;
        }

        public void setType(DiffType type) {
            this.type = type;
        }

        public VALUE getValue() {
            return value;
        }

        public void setValue(VALUE value) {
            this.value = value;
        }

        @Override
        public String toString() {
            return type.toString()+value.toString();
        }
    }
}

```

```

import java.util.List;

public class LcsString extends LongestCommonSubsequence<Character> {
    private String x;
    private String y;

    public LcsString(String from, String to) {
        this.x = from;
        this.y = to;
    }

    protected int lengthOfY() {
        return y.length();
    }
    protected int lengthOfX() {
        return x.length();
    }
    protected Character valueOfX(int index) {
        return x.charAt(index);
    }
    protected Character valueOfY(int index) {
        return y.charAt(index);
    }

    public String getHtmlDiff() {
        DiffType type = null;
        List<DiffEntry<Character>> diffs = diff();
        StringBuffer buf = new StringBuffer();

        for(DiffEntry<Character> entry : diffs) {
            if(type != entry.getType()) {
                if(type != null) {
                    buf.append("</span>");
                }
                buf.append("<span class=\"" + entry.getType().getName() + "\">");
                type = entry.getType();
            }
            buf.append(escapeHtml(entry.getValue()));
        }
        buf.append("</span>");
        return buf.toString();
    }

    private String escapeHtml(Character ch) {
        switch(ch) {
            case '<': return "&lt;";
            case '>': return "&gt;";
            case '"': return "&quot;";
            default : return ch.toString();
        }
    }

    // EXAMPLE. Here's how you use it.
    public static void main(String[] args) {
        LcsString seq = new LcsString("<p>the quick brown fox</p>","<p>the <b>Fast</b> brown dog</p>");
        System.out.println("LCS: " + seq.getLcsLength());
        System.out.println("Edit Dist: " + seq.getMinEditDistance());
        System.out.println("Backtrack: " + seq.backtrack());
        System.out.println("HTML Diff: " + seq.getHtmlDiff());
    }
}

```

C++

```

#include <algorithm>
#include <string>
#include <vector>

#include <stdio.h>

```

```

#include <string.h>

// See http://www-igm.univ-mlv.fr/~lecroq/seqcomp/node4.html.
class LCS
{
    class LCSTable
    {
        size_t    m_;
        size_t    n_;
        size_t*   data_;
    public:
        LCSTable(size_t m, size_t n)
            : m_(m)
            , n_(n)
        {
            data_ = new size_t[(m_ + 1) * (n_ + 1)];
        }
        ~LCSTable()
        {
            delete [] data_;
        }

        void setAt(size_t i, size_t j, size_t value)
        {
            data_[i + j * (m_ + 1)] = value;
        }

        size_t getAt(size_t i, size_t j) const
        {
            return data_[i + j * (m_ + 1)];
        }

        template<typename T> void
        build(const T* X, const T* Y)
        {
            for (size_t i=0; i<=m_; ++i)
                setAt(i, 0, 0);

            for (size_t j=0; j<=n_; ++j)
                setAt(0, j, 0);

            for (size_t i = 0; i < m_; ++i)
            {
                for (size_t j = 0; j < n_; ++j)
                {
                    if (X[i] == Y[j])
                        setAt(i+1, j+1, getAt(i, j)+1);
                    else
                        setAt(i+1, j+1, std::max(getAt(i+1, j), getAt(i, j+1)));
                }
            }
        }
    };

    template<typename T> static void
    backtrackOne(const LCSTable& table,
                const T* X, const T* Y, size_t i, size_t j,
                std::vector<T>& result)
    {
        if (i == 0 || j == 0)
            return;
        if (X[i - 1] == Y[j - 1])
        {
            backtrackOne(table, X, Y, i - 1, j - 1, result);
            result.push_back(X[i - 1]);
            return;
        }
        if (table.getAt(i, j - 1) > table.getAt(i - 1, j))
            backtrackOne(table, X, Y, i, j - 1, result);
        else
            backtrackOne(table, X, Y, i - 1, j, result);
    }

    public:
        template<typename T> static void
        findOne(T* X, size_t m, T* Y, size_t n,
                std::vector<T>& result)
        {
            LCSTable table(m, n);
            table.build(X, Y);
            backtrackOne(table, X, Y, m, n, result);
        }
    };

```

```

    }
};

```

Usage:

```

char X[] = "XMJYAUZ";
char Y[] = "MZJAWXU";
std::vector<char> result;

LCS::findOne(X, strlen(X), Y, strlen(Y), result);
std::string resultString(&result.front(), result.size());

```

Ruby

```

def subsequence(s1, s2)

  return 0 if s1.empty? || s2.empty?

  num=Array.new(s1.size){Array.new(s2.size)}
  s1.scan(/./).each_with_index{|letter1,i|
    s2.scan(/./).each_with_index{|letter2,j|

      if s1[i]==s2[j]
        if i==0||j==0
          num[i][j] = 1
        else
          num[i][j] = 1 + num[i - 1][ j - 1]
        end
      else
        if i==0 && j==0
          num[i][j] = 0
        elsif i==0 && j!=0 #First ith element
          num[i][j] = [0, num[i][j - 1]].max
        elsif j==0 && i!=0 #First jth element
          num[i][j] = [0, num[i - 1][j]].max
        elsif i != 0 && j!= 0
          num[i][j] = [num[i - 1][j], num[i][j - 1]].max
        end
      end
    }
  }
  num[s1.length - 1][s2.length - 1]

end

puts subsequence("houseboat", "computer")

```

JavaScript

```

function LCS(a, b) {
  var m = a.length, n = b.length,
      C = [], i, j;
  for (i = 0; i <= m; i++) C.push([0]);
  for (j = 0; j < n; j++) C[0].push(0);
  for (i = 0; i < m; i++)
    for (j = 0; j < n; j++)
      C[i+1][j+1] = a[i] === b[j] ? C[i][j]+1 : Math.max(C[i+1][j], C[i][j+1]);
  return (function bt(i, j) {
    if (i*j === 0) { return ""; }
    if (a[i-1] === b[j-1]) { return bt(i-1, j-1) + a[i-1]; }
  })
}

```

```

        return (C[i][j-1] > C[i-1][j]) ? bt(i, j-1) : bt(i-1, j);
    }(m, n));
}

```

APL

Computing the length of the LCS

```

C←LCSLength rang;X;Y;i;j;shape;lastalike;words;B;jb
//⌘ Computes the length of the Longest common sequence
X Y←rang

//⌘ Optimize by working with numbers instead of strings
words←X∪Y
X←words⊔X
Y←words⊔Y

//⌘ Optimize and reduce sets by removing similar trailing objects.
shape←(ρX)⊔(ρY)
lastalike←+/∧\φ(shape↑X)=(shape↑Y)
X←lastalike↓X
Y←lastalike↓Y

//⌘ C is a numeric matrix where the height is the shape of X
//⌘ and the width is the shape of Y
C←(1+⌈,ρ`X Y)ρ0

//⌘ Fill C with LCSlengths
j←1+⌈ρY
jb←j-1
:For i :In 1+⌈ρX
    B←X[i-1]=Y
    C[i;B/j]←1+C[i-1;B/jb]
    C[i;(~B)/j]←(∧(B)/C[i;jb])[C[i-1;j]
:EndFor

```

Reading out all LCSs

```

R←MakeDiff rang;do;⊔IO;C;X;Y;i;j;t1;t2
//⌘ Get the diff between two vectors given the LCS in rang1
//⌘ \ret [;1] 1=Added item, 0=Deleted item
//⌘      [;2] index in rang2
//⌘      [;3] index in rang3
//⌘      [;4] The item
⊔IO←0
C X Y←rang

//⌘ Number of elements in each vector. This is the starting
//⌘ point for the calculation below
i j←⌈,ρ`,"X Y

//⌘ Add an empty item before each item... Needed
X Y←' ',`X Y

//⌘ Test 1
t1←{C i j←0 ∨ j>0:{i=0:1 ∨ C[i;j-1]≥C[i-1;j]:1 ∨ 0}0 ∨ 0}

//⌘ Test 2
t2←{C i j←0 ∨ i>0:{j=0:1 ∨ C[i;j-1]<C[i-1;j]:1 ∨ 0}0 ∨ 0}

//⌘ Walk the path and trace additions or removals of items recursively
do←{
    C X Y i jsofar←0
    (i>0)∧(j>0)∧(X[i]=Y[j]):∇ C X Y(i-1)(j-1)sofar
    t1 C i j:∇ C X Y(i)(j-1)(sofar,1 i j(j>0Y))
}

```

```

t2 C i j:∇ C X Y(i-1)(j)(sofar:0 i j(i>X))
sofar
}

//Ⓜ Shoot
R←do C X Y i j(0 4p0)

```

Usage example

```

new←'MZJAWXU'
old←'XMJYAUZ'
C←LCSLength old new
D←MakeDiff C old new
D←⊖D

```

Variable D is:

```

0 1 0 X
1 2 2 Z
0 4 3 Y
1 5 5 W
1 5 6 X
0 7 7 Z

```

J

```

lcs=: [: >./@, [: (* * 1 + >./\@:(_1&|. )@:|: ^:2)^:_ [: ,&0@: ,.&0 =/
'XMJYAUZ' lcs 'MZJAWXU'
4

```

See [1] (<http://www.jsoftware.com/jwiki/Essays/Longest%20Common%20Subsequence>) for details.

Retrieved from "[http://en.wikibooks.org/w/index.php?](http://en.wikibooks.org/w/index.php?title=Algorithm_Implementation/Strings/Longest_common_subsequence&oldid=2770201)

[title=Algorithm_Implementation/Strings/Longest_common_subsequence&oldid=2770201](http://en.wikibooks.org/w/index.php?title=Algorithm_Implementation/Strings/Longest_common_subsequence&oldid=2770201)"

- This page was last modified on 24 February 2015, at 17:51.
- Text is available under the Creative Commons Attribution-ShareAlike License.; additional terms may apply. By using this site, you agree to the Terms of Use and Privacy Policy.